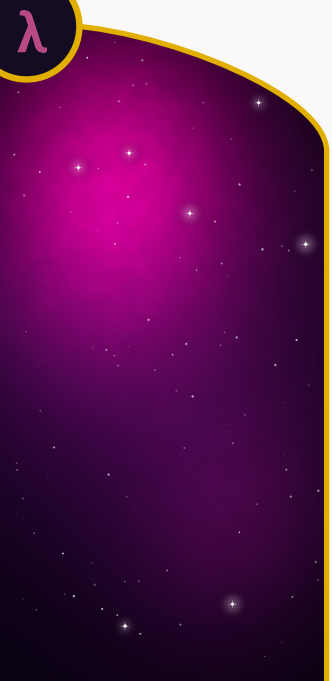


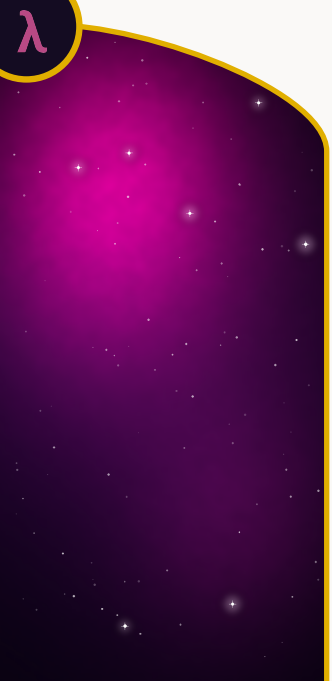
L9B: Lists and Loops II

CS1101S: Programming Methodology

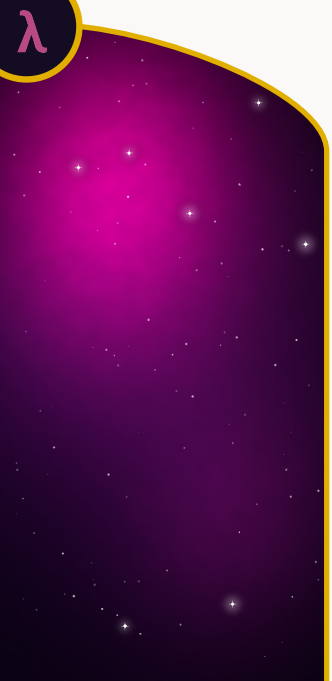
Martin Henz

16th October, 2026

Version 1.2 — Built: 2026-10-16 07:57:57 SGT

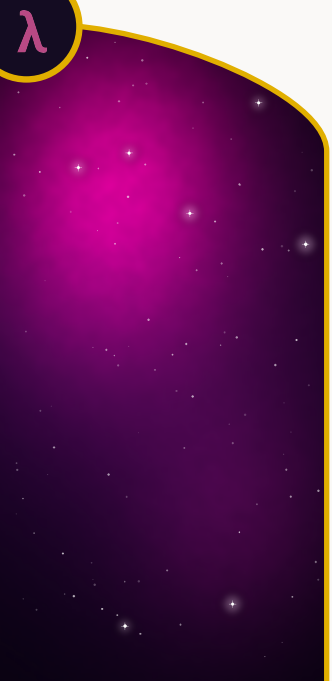


Outline



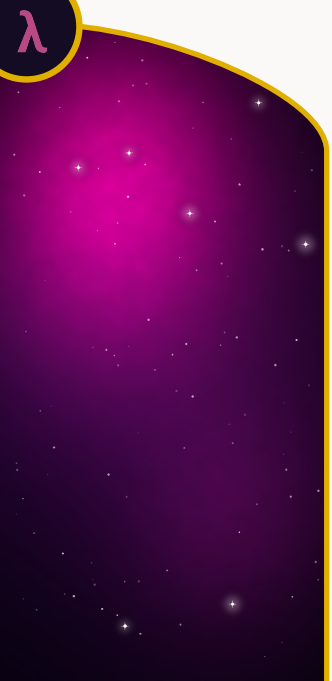
Outline

- Recall: lists give $\Theta(1)$ random access; loops replace recursive traversal



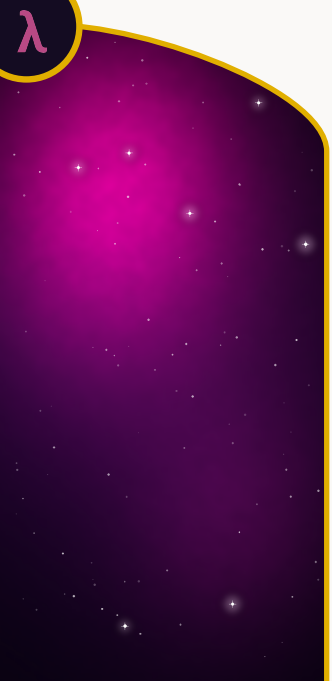
Outline

- Recall: lists give $\Theta(1)$ random access; loops replace recursive traversal
- Warm-up: summing, finding extremes, and comparing lists



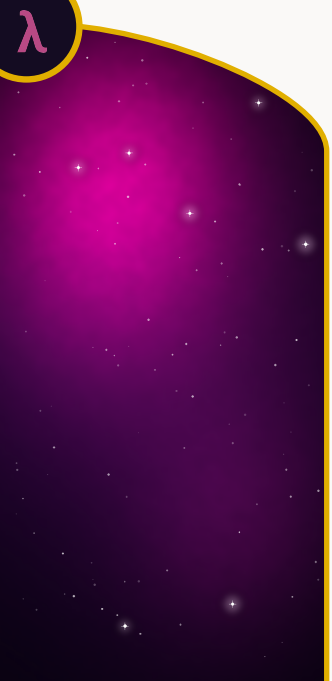
Outline

- Recall: lists give $\Theta(1)$ random access; loops replace recursive traversal
- Warm-up: summing, finding extremes, and comparing lists
- Map and filter, revisited for lists



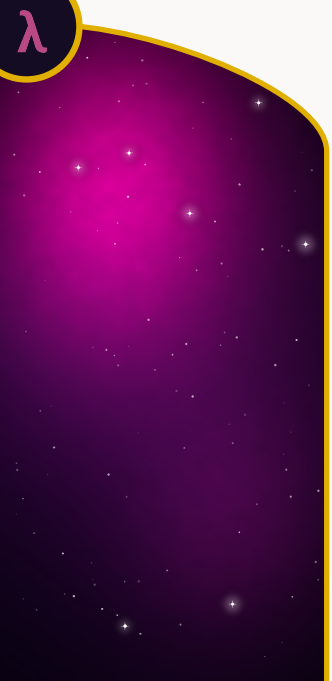
Outline

- Recall: lists give $\Theta(1)$ random access; loops replace recursive traversal
- Warm-up: summing, finding extremes, and comparing lists
- Map and filter, revisited for lists
- Reversing a list in place



Outline

- Recall: lists give $\Theta(1)$ random access; loops replace recursive traversal
- Warm-up: summing, finding extremes, and comparing lists
- Map and filter, revisited for lists
- Reversing a list in place
- `repeat_apply`, revisited with a loop

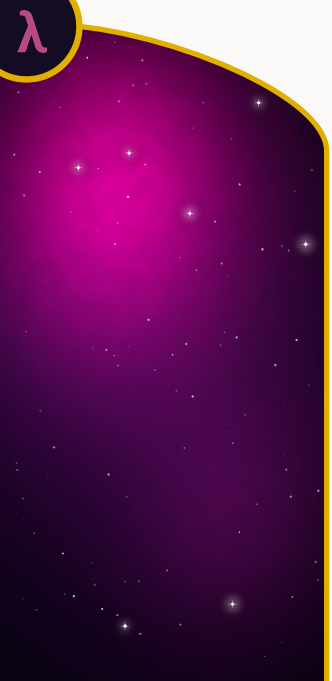


Outline

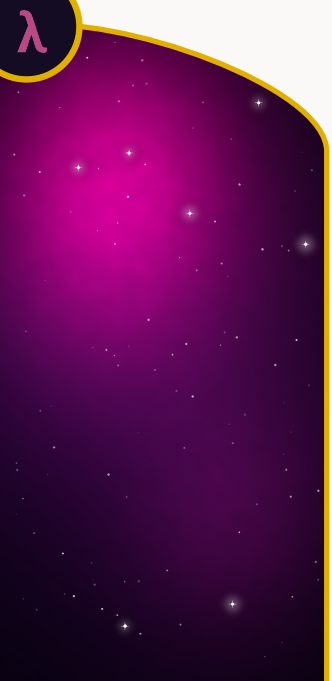
- Recall: lists give $\Theta(1)$ random access; loops replace recursive traversal
- Warm-up: summing, finding extremes, and comparing lists
- Map and filter, revisited for lists
- Reversing a list in place
- `repeat_apply`, revisited with a loop
- Checking whether a list is sorted

Outline

- Recall: lists give $\Theta(1)$ random access; loops replace recursive traversal
- Warm-up: summing, finding extremes, and comparing lists
- Map and filter, revisited for lists
- Reversing a list in place
- `repeat_apply`, revisited with a loop
- Checking whether a list is sorted
- Bonus: revisiting week 1's `stackn` with a list

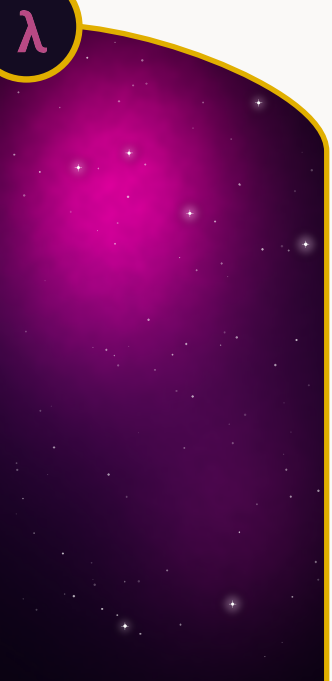


Recall: Lists and Loops



Recall: Lists and Loops

- A list gives $\Theta(1)$ access to any existing element by index.



Recall: Lists and Loops

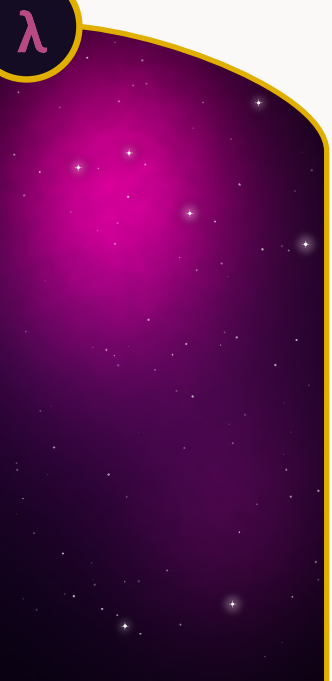
- A list gives $\Theta(1)$ access to any existing element by index.
- `for i in range(len(items))` visits every valid index of `items`.

Recall: Lists and Loops

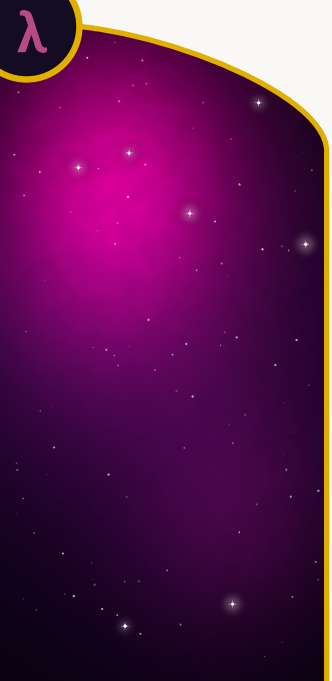
- A list gives $\Theta(1)$ access to any existing element by index.
- `for i in range(len(items))` visits every valid index of `items`.
- Assigning to `items[i]` mutates the list in place; no new list is created.

Recall: Lists and Loops

- A list gives $\Theta(1)$ access to any existing element by index.
- `for i in range(len(items))` visits every valid index of `items`.
- Assigning to `items[i]` mutates the list in place; no new list is created.
- Today: use these two facts to redo problems we first solved with recursion over linked lists.

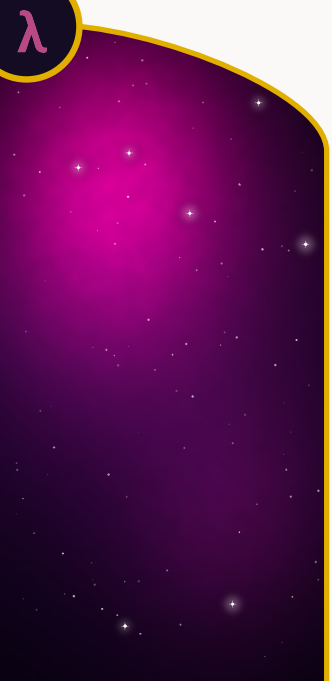


Summing a List



Summing a List

- Recall `llist_sum`: fold over a llist with `reduce`.



Summing a List

- Recall `llist_sum`: fold over a llist with `reduce`.
- The list version accumulates in a loop instead:

Summing a List

- Recall `llist_sum`: fold over a llist with `reduce`.
- The list version accumulates in a loop instead:

```
def list_sum(items):
    total = 0
    for i in range(len(items)):
        total = total + items[i]
    return total

list_sum([3, 1, 5, 2])
```



Summing a List

- Recall `llist_sum`: fold over a llist with `reduce`.
- The list version accumulates in a loop instead:

```
def list_sum(items):
    total = 0
    for i in range(len(items)):
        total = total + items[i]
    return total

list_sum([3, 1, 5, 2])
```

- `total` plays the role `reduce`'s accumulator played before.

Finding the Largest Element



```
def list_max(items):
    largest = items[0]
    for i in range(1, len(items)):
        if items[i] > largest:
            largest = items[i]
    return largest

list_max([3, 1, 5, 2])
```

Finding the Largest Element

```
def list_max(items):
    largest = items[0]
    for i in range(1, len(items)):
        if items[i] > largest:
            largest = items[i]
    return largest

list_max([3, 1, 5, 2])
```

- Start from `items[0]`; the loop only needs to check indices 1 upward.

Finding the Largest Element

```
def list_max(items):
    largest = items[0]
    for i in range(1, len(items)):
        if items[i] > largest:
            largest = items[i]
    return largest

list_max([3, 1, 5, 2])
```

- Start from `items[0]`; the loop only needs to check indices 1 upward.
- `list_min` is the same loop with `>` flipped to `<`.

Finding the Largest Element

```python
def list_max(items):
    largest = items[0]
    for i in range(1, len(items)):
        if items[i] > largest:
            largest = items[i]
    return largest

list_max([3, 1, 5, 2])
```

- Start from `items[0]`; the loop only needs to check indices 1 upward.
- `list_min` is the same loop with `>` flipped to `<`.
- Both are $\Theta(n)$ time, $\Theta(1)$ extra space – one comparison per element.

Comparing Two Lists

```
def lists_equal(a, b):
    if len(a) != len(b):
        return False
    equal = True
    for i in range(len(a)):
        if a[i] != b[i]:
            equal = False
            break
    return equal

lists_equal([1, 2, 3], [1, 2, 4])
```

Comparing Two Lists

```
def lists_equal(a, b):
    if len(a) != len(b):
        return False
    equal = True
    for i in range(len(a)):
        if a[i] != b[i]:
            equal = False
            break
    return equal

lists_equal([1, 2, 3], [1, 2, 4])
```

- `break` stops scanning the moment a mismatch is found; there is no need to check the remaining indices.

Comparing Two Lists

```
def lists_equal(a, b):
    if len(a) != len(b):
        return False
    equal = True
    for i in range(len(a)):
        if a[i] != b[i]:
            equal = False
            break
    return equal

lists_equal([1, 2, 3], [1, 2, 4])
```

- `break` stops scanning the moment a mismatch is found; there is no need to check the remaining indices.
- Best case $\Theta(1)$ (an early mismatch); worst case $\Theta(n)$ (the lists are equal, or differ only at the last index).

Recall: Mapping a List with a Loop



```
def map_list(f, items):
    for i in range(len(items)):
        items[i] = f(items[i])

seq = [3, 1, 5]
map_list(lambda x: 2 * x, seq)
seq
```

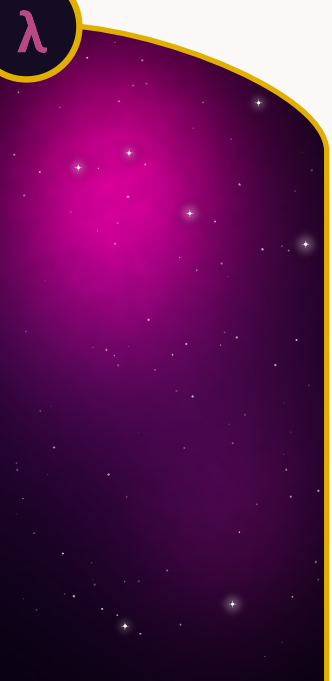
Recall: Mapping a List with a Loop

```
def map_list(f, items):
    for i in range(len(items)):
        items[i] = f(items[i])

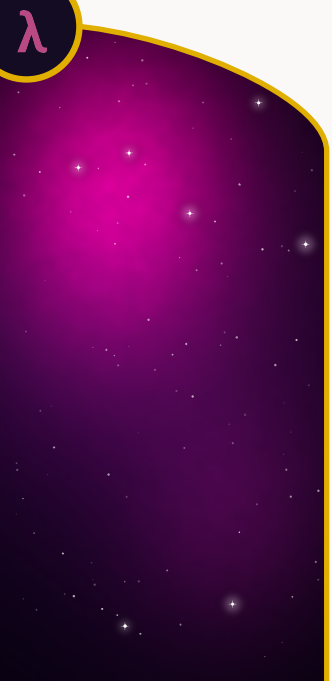
seq = [3, 1, 5]
map_list(lambda x: 2 * x, seq)
seq
```



- From last lecture: `map_list` mutates `items` in place.

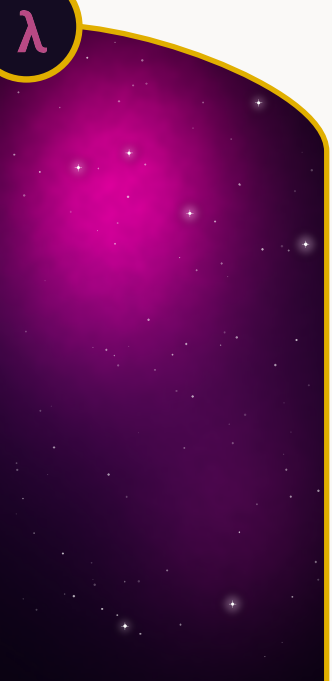


Scaling a List



Scaling a List

- Recall `scale_llist`, written by hand over a llist, then rewritten using `map`.



Scaling a List

- Recall `scale_llist`, written by hand over a llist, then rewritten using `map`.
- The list version reuses `map_list` the same way:

Scaling a List

- Recall `scale_llist`, written by hand over a llist, then rewritten using `map`.
- The list version reuses `map_list` the same way:

```
def scale_list(items, k):
    map_list(lambda x: k * x, items)

seq = [1, 2, 3]
scale_list(seq, 10)
print(seq)
```



Squaring a List

```
def square_list(items):
    map_list(lambda x: x * x, items)

seq = [1, 2, 3, 4]
square_list(seq)
print(seq)
```



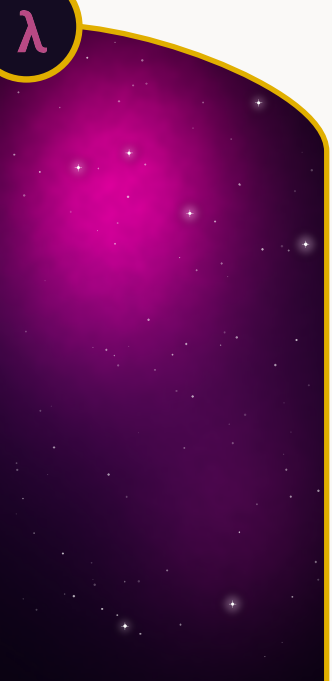
Squaring a List

```
def square_list(items):
    map_list(lambda x: x * x, items)

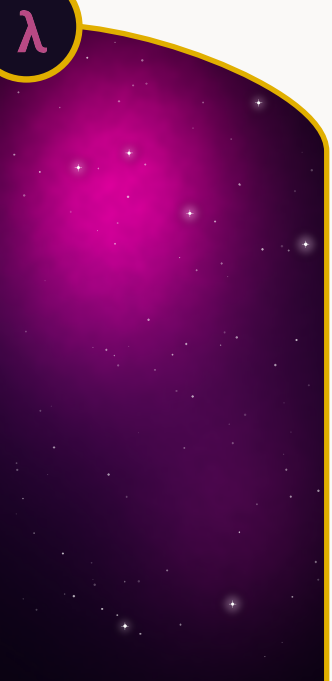
seq = [1, 2, 3, 4]
square_list(seq)
print(seq)
```



- No new pairs, no new list: `map_list` mutates the same list both times.



List Filter: Counting Matches First



List Filter: Counting Matches First

- Recall `even_numbers` via `filter`: each kept element makes a new pair, one at a time.

List Filter: Counting Matches First

- Recall `even_numbers` via `filter`: each kept element makes a new pair, one at a time.
- A list cannot grow one slot at a time – last week's `[None] * n` needs its size `n` up front.

List Filter: Counting Matches First

- Recall `even_numbers` via `filter`: each kept element makes a new pair, one at a time.
- A list cannot grow one slot at a time – last week's `[None] * n` needs its size `n` up front.
- So we make two passes: count matches first, then fill a result list of exactly that size.

List Filter: Counting Matches First

- Recall `even_numbers` via `filter`: each kept element makes a new pair, one at a time.
- A list cannot grow one slot at a time – last week's `[None] * n` needs its size `n` up front.
- So we make two passes: count matches first, then fill a result list of exactly that size.

```
def count_matching(pred, items):
    count = 0
    for i in range(len(items)):
        if pred(items[i]):
            count = count + 1
    return count

count_matching(lambda x: x % 2 == 0, [1, 2, 3, 4, 5, 6])
```

List Filter: Filling the Result List

```
def filter_list(pred, items):
    result = [None] * count_matching(pred, items)
    j = 0
    for i in range(len(items)):
        if pred(items[i]):
            result[j] = items[i]
            j = j + 1
    return result

filter_list(lambda x: x % 2 == 0, [1, 2, 3, 4, 5, 6])
```

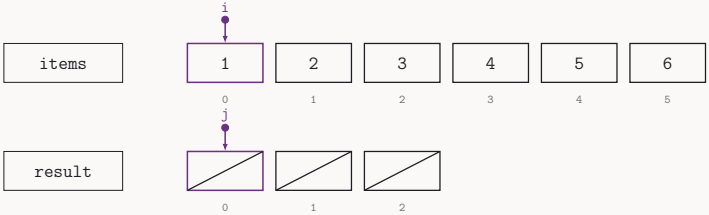
List Filter: Filling the Result List

```
def filter_list(pred, items):
    result = [None] * count_matching(pred, items)
    j = 0
    for i in range(len(items)):
        if pred(items[i]):
            result[j] = items[i]
            j = j + 1
    return result

filter_list(lambda x: x % 2 == 0, [1, 2, 3, 4, 5, 6])
```

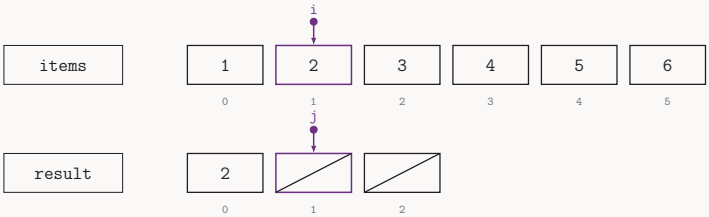
- The second pass walks `items` again, writing each match into the next free slot of `result`.

Watching the Second Pass Run



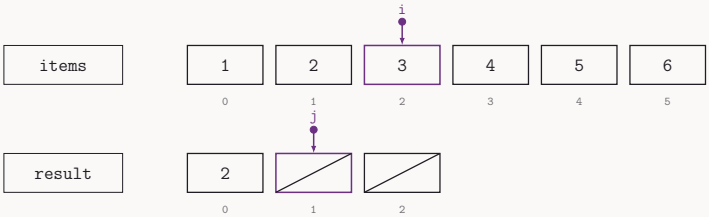
- Click through: `i` scans `items` left to right; `j` only advances when a match gets written into `result`.

Watching the Second Pass Run



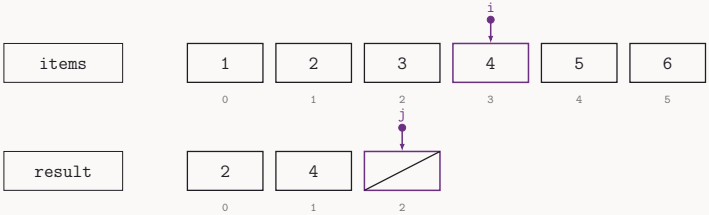
- Click through: `i` scans `items` left to right; `j` only advances when a match gets written into `result`.

Watching the Second Pass Run



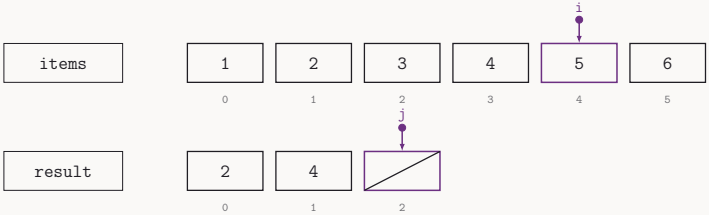
- Click through: `i` scans `items` left to right; `j` only advances when a match gets written into `result`.

Watching the Second Pass Run



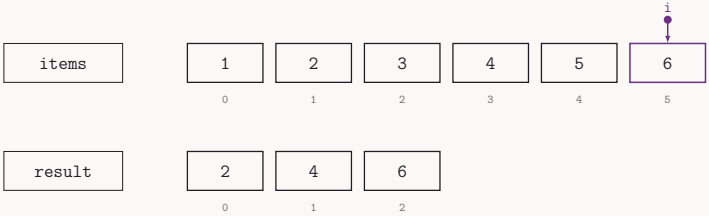
- Click through: `i` scans `items` left to right; `j` only advances when a match gets written into `result`.

Watching the Second Pass Run

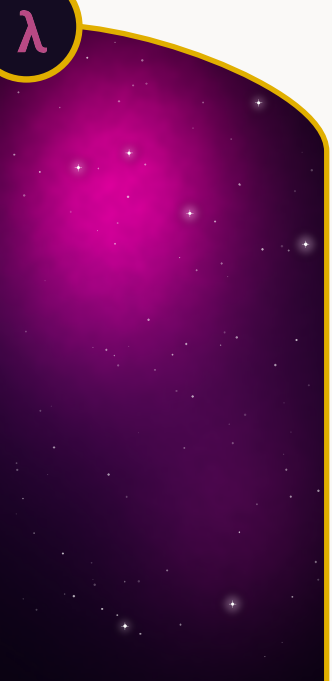


- Click through: `i` scans `items` left to right; `j` only advances when a match gets written into `result`.

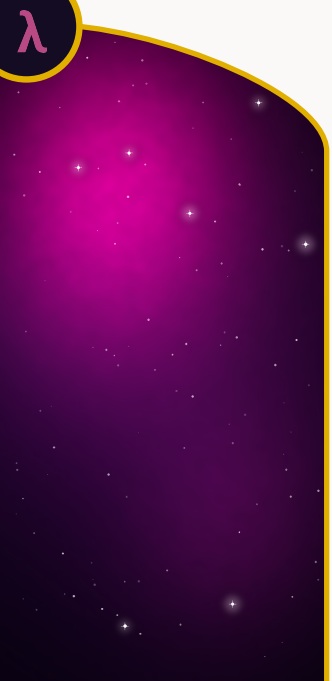
Watching the Second Pass Run



- Click through: `i` scans `items` left to right; `j` only advances when a match gets written into `result`.



Why Two Passes?



Why Two Passes?

- Pass 1 counts how many elements satisfy `pred`: $\Theta(n)$ time.

Why Two Passes?

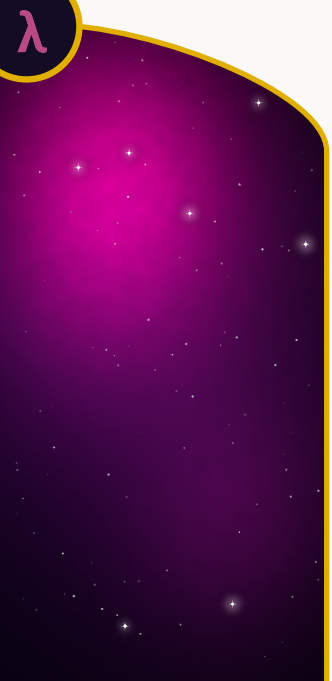
- Pass 1 counts how many elements satisfy `pred`: $\Theta(n)$ time.
- Pass 2 allocates a `count`-sized list and fills it: $\Theta(n)$ time, $\Theta(\texttt{count} + 1)$ space.

Why Two Passes?

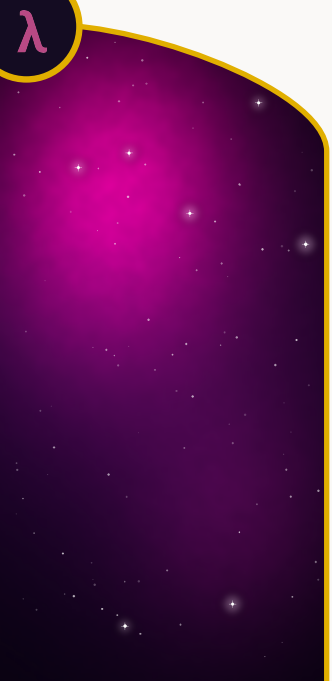
- Pass 1 counts how many elements satisfy `pred`: $\Theta(n)$ time.
- Pass 2 allocates a `count`-sized list and fills it: $\Theta(n)$ time, $\Theta(\texttt{count} + 1)$ space.
- Total: still $\Theta(n)$ time – two passes over n is no worse an order of growth than one.

Why Two Passes?

- Pass 1 counts how many elements satisfy `pred`: $\Theta(n)$ time.
- Pass 2 allocates a `count`-sized list and fills it: $\Theta(n)$ time, $\Theta(\texttt{count} + 1)$ space.
- Total: still $\Theta(n)$ time – two passes over n is no worse an order of growth than one.
- The llist version never needed to know `count` in advance; the list version always does.



Recall: Reversing a Llist



Recall: Reversing a Llist

- `reverse1`: pairs the wrong way round – does not reverse at all.

Recall: Reversing a Llist

- `reverse1`: pairs the wrong way round – does not reverse at all.
- `reverse2`: correct, using `append` – but $O(n^2)$ time.

Recall: Reversing a Llist

- `reverse1`: pairs the wrong way round – does not reverse at all.
- `reverse2`: correct, using `append` – but $O(n^2)$ time.
- `reverse3`: correct and $\Theta(n)$ time, using an accumulator – but every one of its n pairs is newly allocated.

Recall: Reversing a Llist

- `reverse1`: pairs the wrong way round – does not reverse at all.
- `reverse2`: correct, using `append` – but $O(n^2)$ time.
- `reverse3`: correct and $\Theta(n)$ time, using an accumulator – but every one of its n pairs is newly allocated.
- None of the three reverses a llist using its existing pairs.

Reversing a List In Place

```
def reverse_list(items):
    lo = 0
    hi = len(items) - 1
    while lo < hi:
        temp = items[lo]
        items[lo] = items[hi]
        items[hi] = temp
        lo = lo + 1
        hi = hi - 1

seq = [1, 2, 3, 4, 5, 6]
reverse_list(seq)
print(seq)
```

Reversing a List In Place

```
def reverse_list(items):
    lo = 0
    hi = len(items) - 1
    while lo < hi:
        temp = items[lo]
        items[lo] = items[hi]
        items[hi] = temp
        lo = lo + 1
        hi = hi - 1

seq = [1, 2, 3, 4, 5, 6]
reverse_list(seq)
print(seq)
```

- `lo` and `hi` walk inward from both ends, swapping as they go.

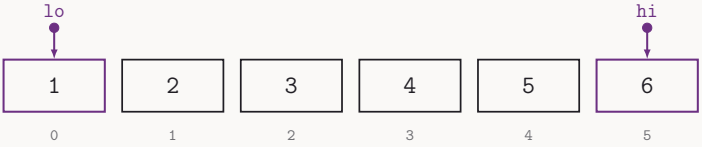
Reversing a List In Place

```
def reverse_list(items):
    lo = 0
    hi = len(items) - 1
    while lo < hi:
        temp = items[lo]
        items[lo] = items[hi]
        items[hi] = temp
        lo = lo + 1
        hi = hi - 1

seq = [1, 2, 3, 4, 5, 6]
reverse_list(seq)
print(seq)
```

- `lo` and `hi` walk inward from both ends, swapping as they go.
- Random access is what makes tracking two indices at once cheap.

Two Pointers, Walking Inward



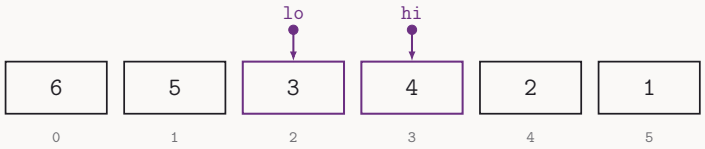
- Click through: each click swaps one pair and steps `lo`/`hi` inward, until they meet and the loop ends.

Two Pointers, Walking Inward



- Click through: each click swaps one pair and steps `lo`/`hi` inward, until they meet and the loop ends.

Two Pointers, Walking Inward

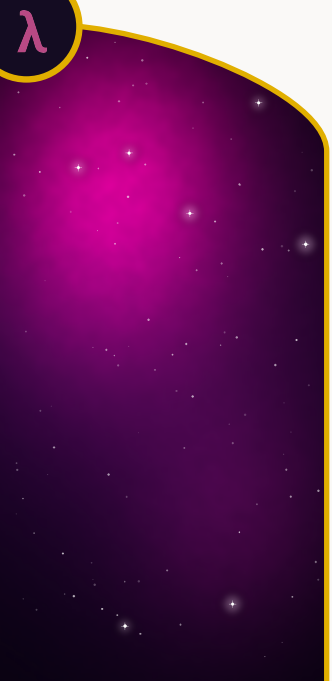


- Click through: each click swaps one pair and steps `lo`/`hi` inward, until they meet and the loop ends.

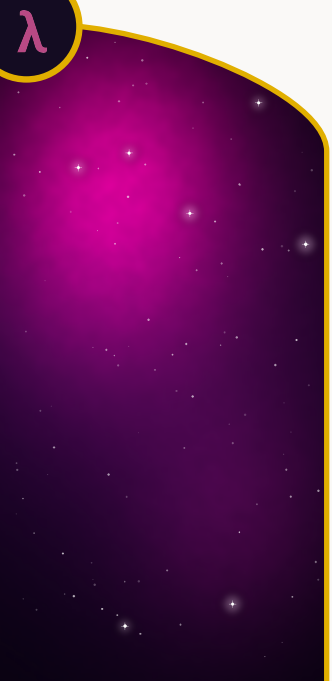
Two Pointers, Walking Inward



- Click through: each click swaps one pair and steps `lo`/`hi` inward, until they meet and the loop ends.



Reverse: Time and Space



Reverse: Time and Space

- The loop body runs $\lfloor n/2 \rfloor$ times: $\Theta(n)$ time, same order of growth as `reverse3`.

Reverse: Time and Space

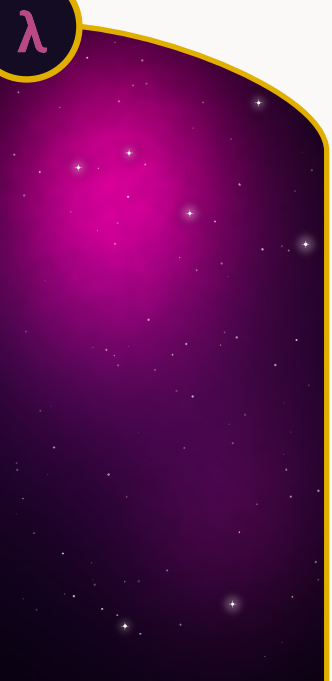
- The loop body runs $\lfloor n/2 \rfloor$ times: $\Theta(n)$ time, same order of growth as `reverse3`.
- Extra space: $\Theta(1)$ – one `temp` variable, no new list.

Reverse: Time and Space

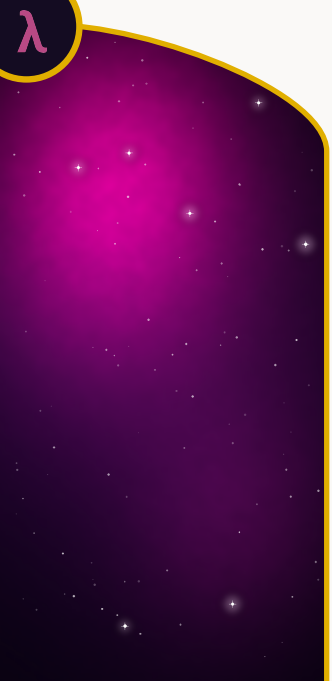
- The loop body runs $\lfloor n/2 \rfloor$ times: $\Theta(n)$ time, same order of growth as `reverse3`.
- Extra space: $\Theta(1)$ – one `temp` variable, no new list.
- Every llist reverse needed $\Theta(n)$ extra space, because pairs cannot be relinked without allocating new ones to hold each new order.

Reverse: Time and Space

- The loop body runs $\lfloor n/2 \rfloor$ times: $\Theta(n)$ time, same order of growth as `reverse3`.
- Extra space: $\Theta(1)$ – one `temp` variable, no new list.
- Every llist reverse needed $\Theta(n)$ extra space, because pairs cannot be relinked without allocating new ones to hold each new order.
- In-place mutation is what closes that gap.

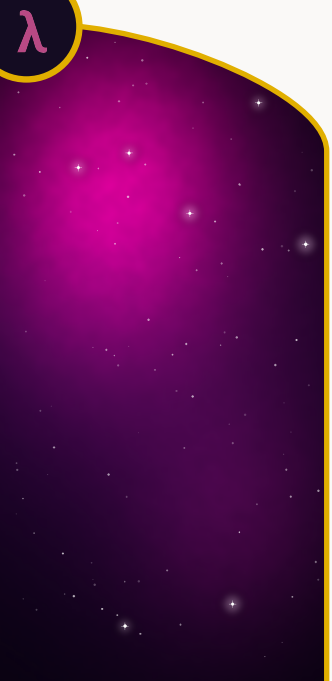


Recall: `repeat_apply`



Recall: `repeat_apply`

- Recall `repeat_apply` from week 2: apply a function `n` times, defined recursively by wishful thinking.



Recall: `repeat_apply`

- Recall `repeat_apply` from week 2: apply a function `n` times, defined recursively by wishful thinking.
- `repeat_apply(pat, n, init) = init` if `n == 0`, otherwise `pat(repeat_apply(pat, n - 1, init))`.

repeat_apply with a Loop

```
def repeat_apply(pat, n, init):
    result = init
    for _ in range(n):
        result = pat(result)
    return result

repeat_apply(lambda x: x * x, 3, 2)
```



`repeat_apply` with a Loop

```
def repeat_apply(pat, n, init):
    result = init
    for _ in range(n):
        result = pat(result)
    return result

repeat_apply(lambda x: x * x, 3, 2)
```

- Same recursion-to-loop pattern as `factorial` – but for repeating any function, not just multiplication.

Checking if a List Is Sorted

```
def is_sorted(items):
    for i in range(1, len(items)):
        if items[i] < items[i - 1]:
            return False
    return True

print(is_sorted([1, 4, 3, 7]))
```

Checking if a List Is Sorted

```
def is_sorted(items):
    for i in range(1, len(items)):
        if items[i] < items[i - 1]:
            return False
    return True

print(is_sorted([1, 4, 3, 7]))
```

- `return False` exits on the first out-of-order pair – no flag variable needed, unlike `lists_equal`'s `break`.

Checking if a List Is Sorted

```
def is_sorted(items):
    for i in range(1, len(items)):
        if items[i] < items[i - 1]:
            return False
    return True

print(is_sorted([1, 4, 3, 7]))
```

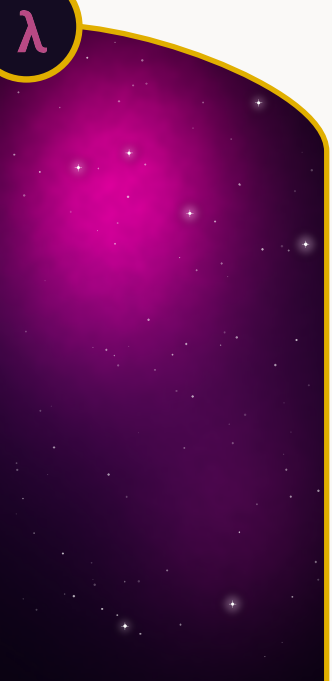
- `return False` exits on the first out-of-order pair – no flag variable needed, unlike `lists_equal`'s `break`.
- $\Theta(n)$ for sorted lists.

Checking if a List Is Sorted

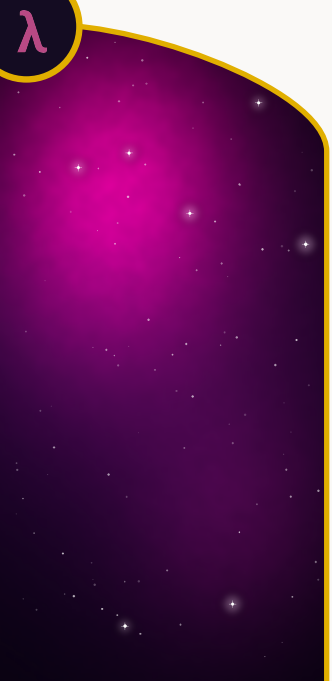
```
def is_sorted(items):
    for i in range(1, len(items)):
        if items[i] < items[i - 1]:
            return False
    return True

print(is_sorted([1, 4, 3, 7]))
```

- `return False` exits on the first out-of-order pair – no flag variable needed, unlike `lists_equal`'s `break`.
- $\Theta(n)$ for sorted lists.
- Binary search assumes the list is sorted – checking it first costs no more than one pass over the list.

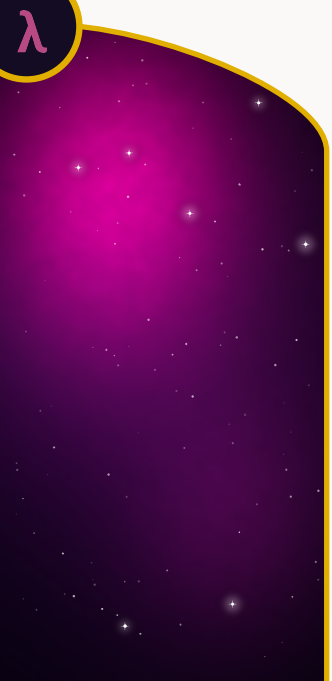


Full Circle: `stackn` Revisited



Full Circle: `stackn` Revisited

- Recall `stackn` from week 1 (Runology): stack `n` copies of a rune, defined recursively by wishful thinking.



Full Circle: `stackn` Revisited

- Recall `stackn` from week 1 (Runology): stack `n` copies of a rune, defined recursively by wishful thinking.
- `stackn(n, rune) = rune` if `n == 1`, otherwise `stack_frac(1/n, rune, stackn(n - 1, rune))`.

Full Circle: `stackn` Revisited

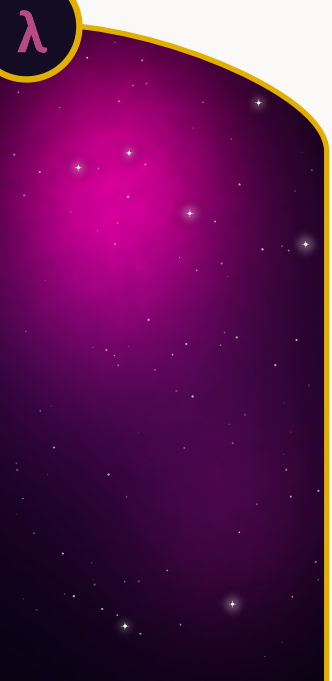
- Recall `stackn` from week 1 (Runology): stack `n` copies of a rune, defined recursively by wishful thinking.
- `stackn(n, rune) = rune` if `n == 1`, otherwise `stack_frac(1/n, rune, stackn(n - 1, rune))`.
- A list can hold the `n` copies; a loop can build the same nested `stack_frac` calls from the inside out.

Stacking Runes with a Loop

```
from rune import show, heart, stack_frac

def stackn(n, rune):
    items = [rune] * n
    result = items[n - 1]
    for i in range(n - 2, -1, -1):
        frac = 1 / (n - i)
        result = stack_frac(frac, items[i], result)
    return result

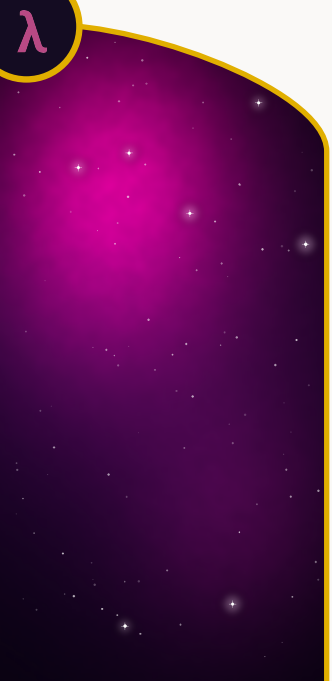
show(stackn(5, heart))
```



Why This Generalizes

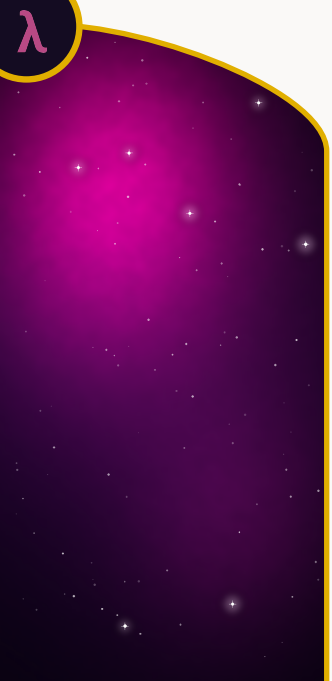
Why This Generalizes

- Every slot of `items` aliases the same rune – safe here, since nothing ever mutates an element (unlike last lecture's zero-matrix warning, which was about mutating shared rows).

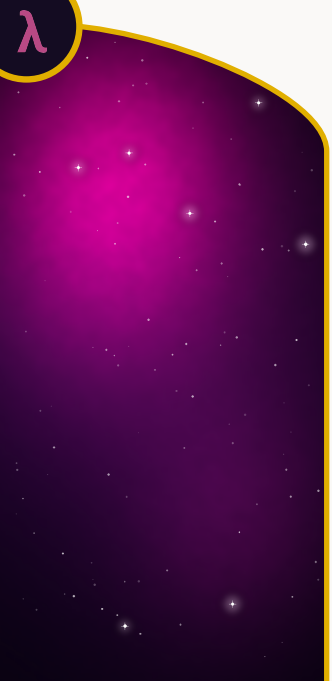


Why This Generalizes

- Every slot of `items` aliases the same rune – safe here, since nothing ever mutates an element (unlike last lecture's zero-matrix warning, which was about mutating shared rows).
- `items` could just as easily hold `n` *different* runes: the loop version stacks any list, not just copies of one rune.



Recap: Lists and Loops II

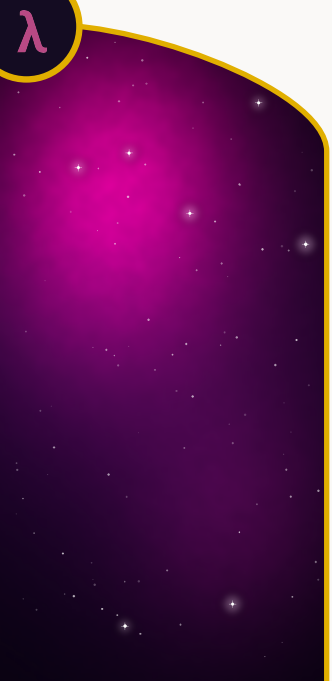


Recap: Lists and Loops II

- A loop over `range(len(items))` replaces recursive traversal, with no call-stack growth.

Recap: Lists and Loops II

- A loop over `range(len(items))` replaces recursive traversal, with no call-stack growth.
- Random access lets an algorithm track more than one position in an list at once – the key idea behind in-place reversal.

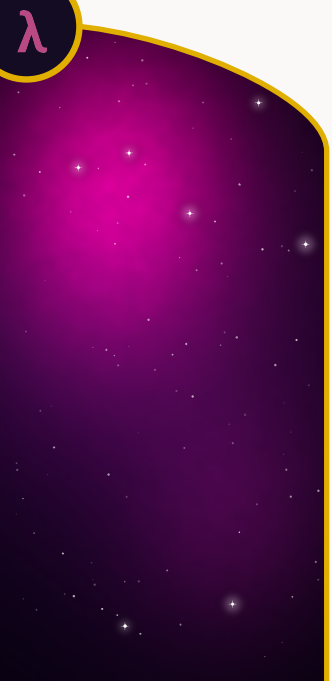


Recap: Lists and Loops II

- A loop over `range(len(items))` replaces recursive traversal, with no call-stack growth.
- Random access lets an algorithm track more than one position in an list at once – the key idea behind in-place reversal.
- Because lists cannot grow, building a derived list (like `filter_list`'s result) needs its size known before allocation.

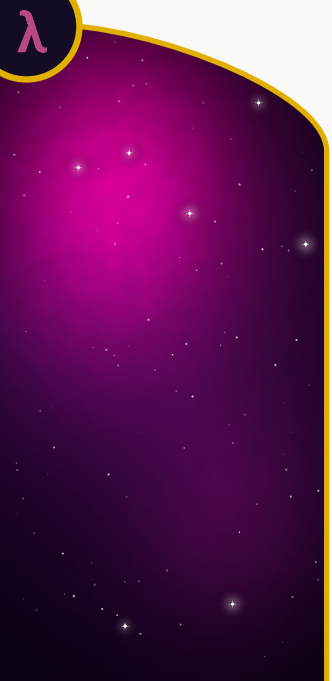
Recap: Lists and Loops II

- A loop over `range(len(items))` replaces recursive traversal, with no call-stack growth.
- Random access lets an algorithm track more than one position in an list at once – the key idea behind in-place reversal.
- Because lists cannot grow, building a derived list (like `filter_list`'s result) needs its size known before allocation.
- `break` and early `return` both let a loop stop as soon as the answer is known.



Recap: Lists and Loops II

- A loop over `range(len(items))` replaces recursive traversal, with no call-stack growth.
- Random access lets an algorithm track more than one position in an list at once – the key idea behind in-place reversal.
- Because lists cannot grow, building a derived list (like `filter_list`'s result) needs its size known before allocation.
- `break` and early `return` both let a loop stop as soon as the answer is known.
- Environment model of `for` and `while` loops



Recap: Lists and Loops II

- A loop over `range(len(items))` replaces recursive traversal, with no call-stack growth.
- Random access lets an algorithm track more than one position in an list at once – the key idea behind in-place reversal.
- Because lists cannot grow, building a derived list (like `filter_list`'s result) needs its size known before allocation.
- `break` and early `return` both let a loop stop as soon as the answer is known.
- Environment model of `for` and `while` loops
- Next week: searching and sorting lists.

Check-In time!

<insert bad pun>

Look for Check-In I9B in the Source Academy!